Specification

Requirement Elicitation Method:

In order to elicit user requirements for an app that will help budget managers keep track of their finances, I have selected an interview-based approach for this project. Interviewing potential users is one of the most efficient and effective methods for gathering qualitative insights from them in a direct manner. A study such as this will be able to uncover in-depth information about the user's preferences, their daily financial behaviors, and their pain points, information that is particularly important for financial applications that need to cater to a diverse range of individual needs. Interviews provide the flexibility to ask open-ended questions about budgeting habits and to probe further into specific areas, such as how users track expenses, what financial tools they are currently using (if any), and where current solutions seem to fall short for users. Budgeting habits are highly personal and often nuanced, and interviews allow you to ask open-ended questions and probe deeper. It is also advantageous to use this method because it facilitates the identification of both explicit and implicit needs (e.g., the ability to set savings goals) which may not be evident when purely survey-based methods are used instead (e.g., anxiety over forgetting bill deadlines). As part of a project such as this one - in which user trust, clarity of information, and usability are all crucial factors - interviews are an important part of making sure that the system design is based on genuine user feedback and practical, real-world scenarios of actual user interactions. In addition, even two or three well-conducted interviews can provide you with a great deal of detail, which means that this method is feasible and useful for an individual project with limited resources and time.

Interview Structure

To gather detailed insights into the budgeting behaviors, needs, and challenges of students, a structured interview approach was employed, targeting ten university students from diverse academic backgrounds and study years. Interview Consent Document The interview was divided into two main sections: quantitative and qualitative. The first section consisted of 15 closed-ended questions designed to capture measurable data on financial habits, tool preferences, and budgeting practices. Questions included topics such as the use of budgeting apps, frequency of checking account balances, preference for mobile versus desktop platforms, and interest in features like expense categorization, reminders, and bank integration. The second section comprised 10 open-ended questions that aimed to elicit deeper, more nuanced information about participants' personal experiences with money management. These questions explored areas such as the challenges students face in staying within budget, desired app features, preferred notification styles, limitations of existing financial tools, and their experience (or lack thereof) with spreadsheets. By combining both types of questions, the interview structure ensured a comprehensive understanding of both the functional needs and emotional expectations users may have from a budget management app tailored to a student demographic. **Interview Results.**

Ethical Considerations

Ethical integrity was a foundational aspect of the interview process, ensuring that all interactions with participants were conducted with respect, transparency, and responsibility. Prior to each interview, participants were provided with a detailed consent form outlining the purpose of the research, what participation involved, how their data would be used, and their rights throughout the process. This included the right to withdraw at any time, skip any questions they felt uncomfortable answering, and request the deletion of their data prior to final analysis. All participants were assured that their responses would remain confidential and be anonymized in any documentation or reporting. The data collected was used solely for academic and developmental purposes in line with university guidelines. Audio recordings (where applicable) were only made with the participant's explicit consent and were securely stored to prevent unauthorized access. By maintaining transparency about the research intent and upholding principles of voluntary participation, informed consent, and data protection, the interview process was conducted in a manner that fully adhered to ethical research standards set by the university and the wider academic community.

Identifiable Patterns within the Interview Data

Following the analysis of both quantitative and qualitative responses from the ten student interviews, several consistent patterns emerged that reflect common financial behaviors, preferences, and challenges faced by students in managing their personal budgets. These patterns not only reveal recurring needs but also highlight key opportunities for feature prioritization in the design of the budget management app. The following bullet points summarize the most prominent patterns identified:

- **High Interest in Notifications and Reminders:**

A majority of participants (9 out of 10) reported frequently forgetting about bills or subscription deadlines and expressed a strong desire for timely reminders via mobile notifications.

- **Preference for Mobile Platforms:**

Most students reported frequently forgetting about bills or subscription deadlines and expressed a strong desire for timely reminders via push notifications.

- **Desire for Visual Representations of Data:**

All participants indicated that visual tools such as graphs, charts, and dashboards would make it easier for them to understand their spending habits and track financial goals.

- **Low Current Usage of Budgeting Tools Despite Interest:**

Although few students currently use a budgeting app, nearly all showed interest in using one if it was user-friendly, automated, and tailored to student needs.

- **Recurring Overspending and Impulse Purchases:**

Overspending was a common issue, with many participants stating they often exceed their intended budgets due to lack of real-time tracking or small purchases adding up unnoticed.

- **Clear Demand for Expense Categorisation and Goal Setting:**

Participants frequently mentioned the need to separate “needs” vs. “wants” and set specific savings or spending goals, showing a preference for structured budgeting.

- **Mixed Views on Automated Entry vs. Manual Entry:**

Some students preferred automatic syncing with bank accounts, while others favored manual entry for greater control - suggesting the app should support both methods.

- **Minimal Use of Spreadsheets or Traditional Tools:**

Many students found spreadsheets tedious or difficult to maintain, highlighting a gap that a well-designed app could fill by offering a more engaging and intuitive experience.

These patterns will directly inform the system and user requirements in the next section and ensure that the app is designed to meet the real-world expectations and limitations of student users.

Analysis of Quantitative and Qualitative Data

The combined analysis of the quantitative and qualitative interview responses provided a comprehensive understanding of student budgeting behaviors, technological preferences, and unmet needs. The quantitative data helped identify clear behavioral trends — for example, the majority of students (90%) preferred using a mobile app for budgeting, and all participants agreed that visual aids such as graphs and charts were essential for understanding their finances. Responses also showed that while only a few currently use budgeting tools, most students face common challenges such as forgetting bills, overspending, and a lack of financial structure. A significant proportion (over 80%) supported features like reminders, daily spending limits, and goal-setting. These insights helped define what features are not only useful but expected from a modern budgeting solution. On the other hand, the qualitative responses enriched the analysis by offering personal perspectives and emotional contexts behind these behaviors. Students expressed frustrations such as the tediousness of spreadsheets, difficulty staying motivated, or finding existing tools overly complex or unengaging. They also provided constructive feature suggestions like categorizing expenses into “needs vs. wants,” having customizable alerts, and even sharing budgets with housemates. The qualitative data revealed nuanced expectations — students don’t just want functionality, they want simplicity, automation, and personalization. Together, these two data sets validated and complemented each other, forming a reliable foundation for defining user-driven system requirements and guiding user-centered design decisions.

Synopsis of User Requirements collected for the system

Functional User Requirements (FUR) & System Requirements

Functional User Requirements (FUR)

User Requirement 1: The user shall be able to track their daily and monthly expenses.

Justification: According to Q!, only 5 out of 10 students currently track their expenses. In Q8, most students admitted to overspending often or sometimes, suggesting that tracking is essential to improving financial control.

User Requirement 2: The user shall receive reminders for upcoming bills or subscriptions.

Justification: Q5 revealed that 9 out of 10 students forget bill deadlines, and Q6 showed unanimous interest in receiving notifications or reminders for payments.

User Requirements 3: The user shall be able to categorise their expenses into “needs” and “wants”.

Justification: As indicated in Q10, all 10 students expressed a desire to distinguish between essential and non-essential spending categories.

User Requirement 4: The user shall be able to set a daily spending limit and receive warnings when close to it.

Justification: In Q9, 9 out of 10 participants said a daily limit feature would be helpful in keeping them accountable and avoiding impulse purchases.

User Requirement 5: The user shall be able to set and monitor monthly savings goals.

Justification: Q14 demonstrated that 100% of participants were interested in using budgeting goals, such as saving a specific amount each month.

User Requirement 6: The user shall view spending trends through visual charts and graphs.

Justification: Q7 confirmed that all students prefer visual aids (like graphs and charts) to better understand their financial patterns and make informed decisions.

User Requirement 7: The user shall choose between manual and automated tracking of expenses.

Justification: Q11 showed a 50/50 split with half preferring automated tracking via bank integration, while the other half preferred manual entry to maintain control. This supports a flexible dual-option design.

User Requirement 8: The user shall have the option to integrate their bank account for real-time updates.

Justification: Q12 revealed that 5 out of 10 students wanted banking integration. This indicates it should be optional - appealing to users who value automation while preserving privacy for others.

User Requirement 9: The user shall be able share budget data with a trusted person (e.g. flatmate).

Justification: In the qualitative section (Q7), several students, particularly those in shared living situations, expressed interest in sharing budgets or expenses with housemates or partners.

User Requirement 10: The user shall receive a summary of their spending each week or month.

Justification: Responses to Q6 showed interest in tracking spending habits over time, and in Q15, participants overwhelmingly agreed that budgeting reduces financial stress. Weekly/monthly summaries can facilitate this reflective process.

Functional System Requirements (FSR)

System Requirement 1: The system shall allow users to manually input daily expenses and categorise them.

Justification: In Q11, 5 students preferred manual expense tracking. Additionally, qualitative answers revealed that some users feel more in control when entering data themselves rather than relying on automation.

System Requirement 2: The system shall send automated notifications based on upcoming bill dates and subscription cycles.

Justification: According to Q5, 9 out of 10 students forget about bills. Q6 showed 100% of students want reminders, making notification functionality essential.

System Requirement 3: The system shall allow users to tag expenses as “needs” or “wants”.

Justification: Q10 found all students wanted to separate essential spending from discretionary spending. This tagging supports more accurate analysis and mindful budgeting.

System Requirement 4: The system shall enable users to set custom daily spending limits and track progress in real time.

Justification: As shown in Q9, nearly all students (9 out of 10) support a feature for daily spending limits, which helps them prevent overspending from creeping up unintentionally.

System Requirement 5: The system shall allow users to create, edit, and track monthly savings goals.

Justification: In Q13, all students indicated they would use a budgeting goal feature such as saving a set amount each month, making this functionality central to the app’s value.

System Requirement 6: The system shall generate dynamic visualisations (bar charts, pie charts) of spending and saving trends.

Justification: Based on Q7, all 10 participants prefer visual data representation, which allows them to engage more effectively with their financial behaviour.

System Requirement 7: The system shall allow users to toggle between manual expense entry and bank account syncing.

Justification: Q11 confirmed mixed preferences: some students want the speed of automation, while others prefer manual control - the system must support both workflows.

System Justification 8: The system shall integrate securely with external banking APIs for live transaction imports.

Justification: Q12 showed that 5 out of 10 students were interested in syncing their bank data, indicating that integration should be available for those who opt-in.

System Justification 9: The system shall allow controlled sharing of budget summaries or categories with other registered users.

Justification: From qualitative Q7, users who live with others expressed a need to share specific budget areas, such as utility expenses, with flatmates or family members.

System Requirement 10: The system shall compile and send weekly/monthly summaries via email or app notifications.

Justification: In Q6, most users said monthly comparisons would be helpful, and in Q15, all agreed that budgeting reduces stress - regular summaries reinforces positive financial habits.

Non-Functional User Requirements (NFUR)

User Requirement 1: The user expects the app to be easy to use with minimal learning curve.

Justification: In the qualitative responses (Q3, Q9), multiple students said they abandoned budgeting tools or spreadsheets because they were too “complex,” “boring,” or “hard to maintain.”

User Requirement 2: The user expects their financial data to remain private and secure.

Justification: Q12 showed that only half of the students were willing to integrate their bank accounts, citing privacy and control concerns. This indicates data confidentiality is a high priority.

User Requirement 3: The user wants to access the app on mobile devices with a full functionality.

Justification: In Q3, 9 out of 10 participants preferred using a mobile app over a desktop tool, underscoring the need for responsive mobile access.

User Requirement 4: The user wants the app to be responsive and not lag or crash.

Justification: Although not directly asked in a quantitative question, students commented in qualitative answers (Q1, Q3) that they don’t stick with tools that are sluggish or feel “clunky”.

User Requirement 5: The user expects the app to work even when offline for basic expense entry.

Justification: In qualitative feedback (Q5, Q9), several students mentioned issues with app availability or internet access and indicated that offline input would be helpful.

Non-Functional System Requirements (NFSR)

System Requirement 1: The system should respond to all user actions within 2 seconds.

Justification: Qualitative frustrations with “slow” or “unreliable” tools (Q1, Q9) suggest users expect a smooth, real-time interaction experience.

System Requirement 2: The system must store user data securely, adhering to GDPR and university ethical standards.

Justification: As evident from mixed responses to Q12, many students are cautious about sharing sensitive financial data, demanding robust data protection policies.

System Requirement 3: The system interface must be mobile-friendly and intuitive for users with varying levels of technical skill.

Justification: Based on Q3, 90% of students want a mobile-first solution. Additionally, qualitative responses revealed that several users avoid “overwhelming” apps.

System Requirement 4: The app should support offline access for manually viewing and entering expenses.

Justification: Referencing NFUR5 and qualitative answers (Q5), students value offline usability during periods without stable internet.

System Requirement 5: The system must encrypt all financial data both in transit and at rest.

Justification: Inferred from Q12, half of participants hesitated about banking integration due to privacy risks, Encryption helps mitigate those concerns.

System Requirement 6: The app should require no more than 3 user actions to access key budgeting features.

Justification: Qualitative answers (Q1, Q9) repeatedly emphasised that students value speed and simplicity - fewer steps means better retention and usability.

System Requirement 7: The system should have >99% uptime during typical student usage hours.

Justification: Though not asked directly, the expectation for reliability implied in users’ needs for reminders, summaries, and daily tracking (Q2, Q6, Q10).

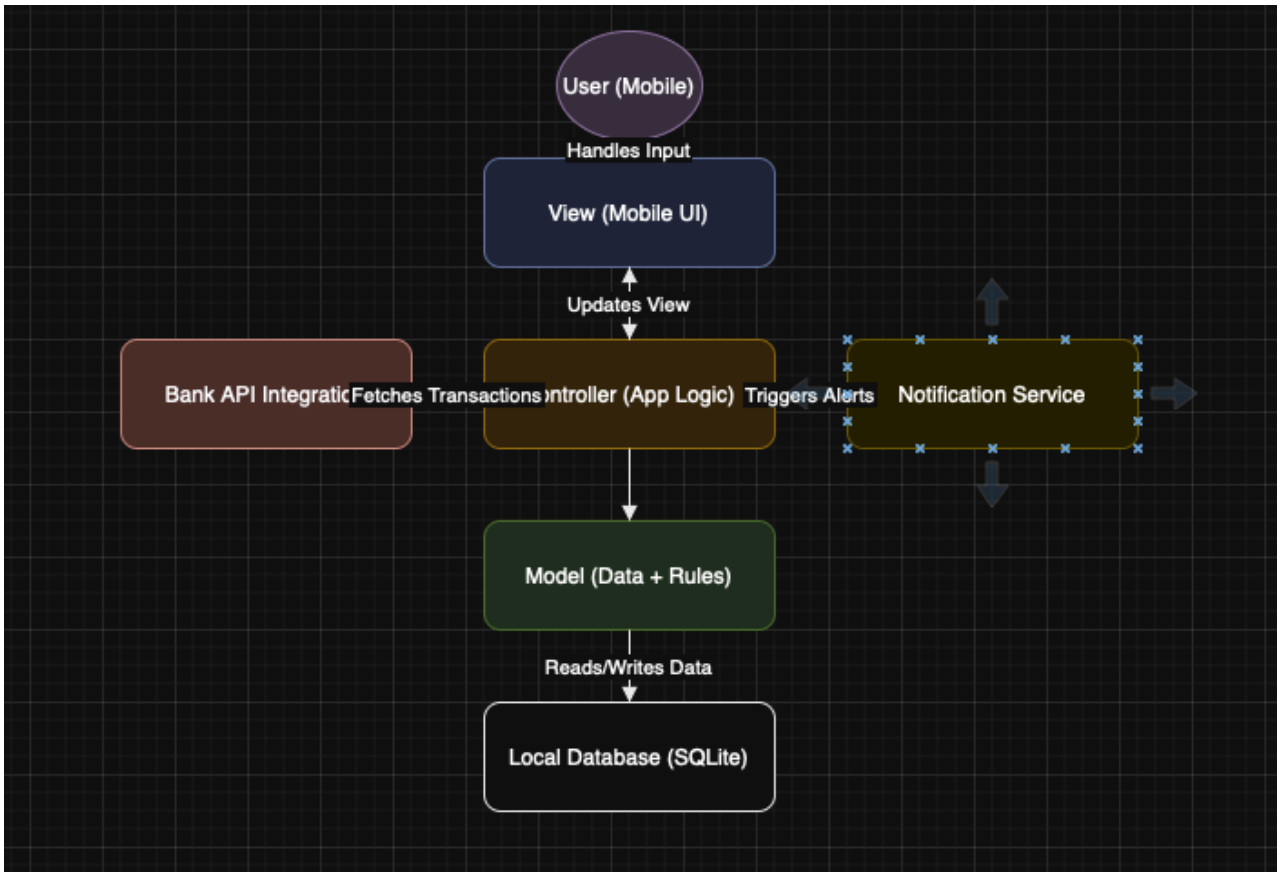
Design

Architectural Design

In designing the system architecture for the budget management application, several architectural patterns were evaluated to determine which would provide the best balance of modularity, usability, and support for user interaction. The architectures considered included Layered, Hierarchical, Event-Based, and Model-View-Controller (MVC). The Layered architecture offers strong modularity by organizing the application into tiers such as presentation, logic, and data layers, which can simplify development and testing. However, its strict top-down communication flow can lead to inefficiencies and makes it less suitable for highly interactive applications like this one, where users need real-time feedback and quick visual updates. Hierarchical architecture, typically seen in command-control systems, establishes a tree-like relationship between components, which works well for fixed control structures but becomes rigid and inefficient in user-centric applications that require dynamic two-way interactions. On the other hand, Event-Based architecture is ideal for systems requiring asynchronous communication, such as real-time updates and notifications. While its loose coupling between components supports scalability, it can introduce complexity in tracking the flow of data and debugging interactions—making it unnecessarily elaborate for a lightweight, student-focused budgeting tool.

Ultimately, the architecture that best supports the needs of this app is Model-View-Controller (MVC). MVC is a proven architectural pattern that separates the application into three interrelated components: the Model, which handles data and logic; the View, responsible for presenting information to the user; and the Controller, which interprets user input and updates both the Model and the View. This structure is ideal for applications like this one, where users frequently input data (e.g., expenses), view results (e.g., graphs and summaries), and expect quick feedback. MVC also aligns perfectly with the data-oriented nature of the budgeting app. The Model can efficiently manage core financial entities such as expenses, categories, savings goals, and budget limits. The Controller processes user actions—like adding a transaction or updating a limit—and passes the resulting state back to the View, which reflects changes visually and in real time. Furthermore, MVC's separation of concerns supports iterative development and testing, making it easier to maintain and expand the app as needed. Considering the requirement for a mobile-friendly, interactive, and user-driven system, MVC stands out as the most appropriate and scalable architectural choice for this project.

Architectural Design Diagram



The architectural diagram represents the core structure and interaction flow of the budget management application, designed using the Model-View-Controller (MVC) framework. At the top of the diagram is the User, representing students who interact with the application primarily via a mobile interface. The View layer is the visual representation of the system — responsible for displaying charts, input forms, summaries, and goal progress. User inputs are passed from the View to the Controller, which acts as the central logic unit, interpreting actions such as adding expenses, setting limits, or modifying savings goals. The Controller then communicates with the Model, which encapsulates the business logic and manages the application's core data structures — including expenses, categories, budgets, and user settings. The Model reads and writes data to the Local Database (such as SQLite), ensuring persistence across sessions. Additionally, the Controller interacts with two auxiliary components: the Notification Service, which sends alerts and summaries based on financial activity, and the Bank API Integration Module, which allows for optional synchronization of bank transactions. Each component is clearly color-coded to highlight its architectural role, and directional arrows indicate the flow of control and data between components. Overall, the diagram illustrates a modular, scalable, and interactive system architecture tailored for student budget management needs.

Use Case Tables

Use Case 1: Add an Expense

Element	Description
Actor	User
Description	The user adds a new expense to their daily spending log.
Pre-condition	User is logged in and on the expense entry screen,
Post-Condition	Expense is recorded, and budget data is updated.
Data	**Input:** Amount, category, date, note **Output:** Confirmation, updated dashboard
Valid Case	User enters £20, selects “Groceries”, adds note, clicks “Save” → Entry saved.
Error case	User leaves the amount field blank → App displays error message and prevents save.

Use Case 2: Set a Daily Spending Limit

Element	**Description**
Actor	User
Description	The user sets a maximum spending amount for the current day.
Pre-condition	User is logged in and has permission to change settings.
Post-condition	Spending limit is stored and tracked throughout the day.
Data	**Input:** Limit amount (e.g., £30) **Output:** Confirmation message, threshold tracking
Valid Case	User enters £30, clicks “Set Limit” → App confirms the limit is active.
Error Case	User enters a negative number → App shows validation error and blocks action.

Use Case 3: View Monthly Budget Summary

Element	Description
Actor	User
Description	The user views a breakdown of income, expenses, and savings for the month.
Pre-condition	User has logged at least one month of data.
Post-condition	Monthly summary is displayed with charts.
Data	**Input:** Request for current month summary **Output:** Bar/pie charts, totals
Valid Case	User selects “June Summary” → App loads a graph with totals for income expenses.
Error Case	No data available for selected month → App shows “No data found” message

Use Case 4:

Element	Description
Actor	User
Description	Users are notified about upcoming bill payments via push notification.
Pre-condition	User has entered bill info and allowed notifications.
Post-condition	Users receive a notification and may take action.
Data	**Input:** Bill due date, name, amount **Output:** Notification content
Valid Case	User receives notification: “Your £50 internet bill is due tomorrow.”
Error Case	Notification system fails → No reminder is sent, potentially missing bill.

Use Case 5: Categorise an Expense as “Need” or “Want”

Element	Description
Actor	User
Description	Users assign an expense as a “need” or a “want” for tracking.
Pre-condition	Expense has been logged or is being created.
Post-condition	Expense is tagged and reflected in analytics.
Data	**Input:** Expense, selection (Need/Want) **Output:** Updated tag & category view
Valid Case	User logs £15 movie ticket and tags it as “Want” → Expense categorised.
Error Case	User skips category selection → App assigns default or shows prompt.

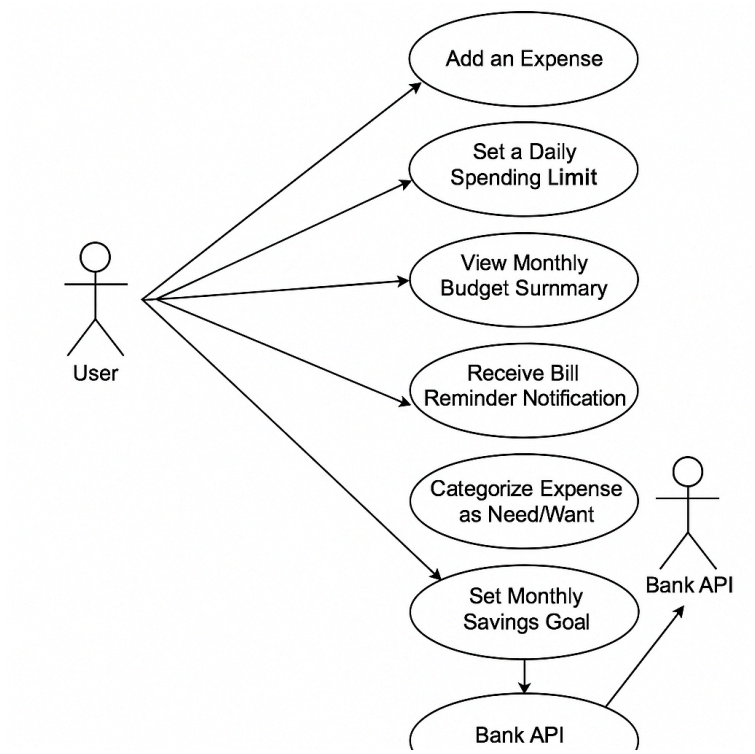
Use Case 6: Integrate Bank Account for Auto-Tracking

Element	Description
Actor	User, Bank API
Description	User links a bank account to import transactions automatically.
Pre-condition	User has valid bank credentials and consents to link.
Post-condition	App fetches and syncs with bank transaction feed.
Data	**Input:** Login credentials, bank selection **Output:** Imported transaction list
Valid Case	User successfully links “Bank Account” → App imports 20 transactions.
Error Case	Incorrect credentials → App displays “Login failed” error.

Use Case 7: Set Monthly Savings Goal

Element	Description
Actor	User
Description	User sets a financial goal to save a fixed amount by month-end.
Pre-condition	Users have budget flexibility and savings features enabled.
Post-condition	Goal is saved, and tracked in the progress view.
Data	**Input:** Goal amount (e.g., £200) **Output:** Confirmation via a progress bar.
Valid Case	User sets a goal of £200 → App tracks contributions and displays progress.
Error Case	Goal exceeds available funds → App shows warning and blocks submission.

Use Case Diagram



Challenges with Designing the Architectural Design

Designing the architectural structure for my budget management application came with several challenges that required careful thought and iteration. One of the first obstacles I encountered was choosing an architecture that struck the right balance between simplicity for development and scalability for future improvements. I explored different models—Layered, Hierarchical, Event-Based, and Model-View-Controller (MVC)—each with their own strengths and weaknesses. It wasn't just about picking the most popular approach; I had to evaluate how each one aligned with the core needs of my app, like regular user input, real-time budget tracking, visual data representation, and integration with external banking APIs. I found that some architectures, while structured, lacked the flexibility needed for a mobile-first app that relies heavily on dynamic user interaction. Settling on the MVC pattern was not an immediate decision, but it proved to be the most effective because it allowed me to separate data, logic, and UI in a way that made sense for both development and user experience. Even then, adapting MVC to support components like push notifications and optional banking integration required thoughtful planning. Translating all of this into a clear architectural diagram was another challenge—I had to visualize how components would interact, what data would be shared, and how to represent optional features without cluttering the overall design. Overall, the process pushed me to think critically about system structure, user needs, and long-term maintainability, and ultimately helped me build a solid architectural foundation for the project.

Implementation

Version Control

In the case of the Budget Management App, I will be utilising GitHub for version control. GitHub is the most seamless and accessible option which will be optimal for the Budget Management App with its simple to understand & intuitive interface, branch management tools, and in-app integrations with multiple IDEs like Visual Studio Code. Additionally, GitHub makes it easy to track changes & roll back errors. With the commit history, visual diff tracking and built-in tracking features as well as safely creating and storing backups of my code, GitHub will be the most convenient and efficient tool to utilize for the Budget Management App.

Technologies and Languages

For the development of my budget management app, I will be choosing to use Flutter for the frontend interface and Dart for the backend logic. Flutter's widget-based structure made it easier to design a clean and responsive UI, while Dart integrates seamlessly within the Flutter ecosystem, allowing for a unified and efficient development process. To manage data effectively, I've decided to use a hybrid approach involving both Firebase and Hive. Firebase is used as the cloud database, enabling users to sync their data across multiple devices. This ensures that their financial analytics such as expense history, savings goals, and bank integrations can be accessed from anywhere in real time. To complement Firebase, I implemented Hive as the local storage solution. Hive is a lightweight, high-performance database that allows me to store device-specific data locally, such as temporary user preferences, cached reports, and offline expense entries. This setup not only improves performance by reducing cloud dependency but also ensures data availability even when the user is offline. The combination of Firebase and Hive provides a robust and scalable infrastructure, making the app reliable, efficient, and responsive across both iOS and Android devices.

Managing Issues and Bugs

I will implement a proactive strategy for managing bugs and issues by utilising an in-app feedback mechanism, allowing users to directly report any bugs or unexpected behaviours they encountered. This should provide real-time insight into issues from the user's perspective and allow me to prioritise fixes based on actual usage. Alongside this, regular testing routines, including automated tests will be done for critical components such as expense entry, budget calculations, and data syncing ensuring regressions are caught early, especially when implementing new features. For example, during development, I introduced a new savings goal feature and later discovered through automated tests that setting a savings target higher than a user's total income didn't trigger a warning or validation error. This was a potential usability flaw that could confuse users, so I was able to quickly implement a fix before releasing the update. By combining real-user feedback with structured testing practices, I was able to detect, understand, and resolve issues efficiently ultimately improving the user experience and the app's reliability over time.

Devices Targeted

When planning the deployment of my budget management app, I made it a priority to consider the wide variety of devices that users might access it from. Since the general population is highly tech-centric, with smartphones and tablets being the most commonly used platforms, I chose to prioritise iOS devices, especially iPhones, as the primary target. These devices are widely adopted and offer a consistent environment for app performance. However, I also aimed for broader accessibility by ensuring the app could function smoothly on iPads and other non-iOS devices such as Android smartphones and tablets. In addition, I used a desktop build primarily for internal testing and debugging, which allowed me to quickly check UI behaviour and logic without the need for continuous mobile deployment. One of the most important technical considerations was responsiveness and adaptability across screen sizes. I used Flutter's flexible layout system to ensure that the UI components could scale and reflow gracefully across various screen resolutions and device orientations. This approach allowed me to deliver a consistent and intuitive user experience, whether the app was being accessed on a compact smartphone, a larger tablet, or even during desktop testing sessions.

Testing

Testing played a vital role in ensuring that my budget management app functioned reliably and aligned with both user expectations and the functional requirements outlined during the early stages of development. I began with unit testing, which focused on validating the accuracy of individual components. For instance, I wrote test cases to ensure that when a user added a new expense, the app correctly updated the total monthly spending and triggered alerts if a predefined budget limit was exceeded. I also accounted for edge cases, such as handling incomplete or missing data. One example involved simulating a scenario where a user attempted to submit an expense entry without entering an amount—testing ensured the app would return a validation error rather than crashing or saving faulty data. In addition to core logic testing, I conducted thorough User Acceptance Testing (UAT) by engaging the same interview participants to trial the app and provide feedback based on their expectations and original input. Their insights were incredibly valuable in refining the user experience and ensuring the app addressed real-world needs. I also performed detailed UI Testing to confirm that the interface components rendered correctly across different devices and operating systems. This included verifying that graphs displayed properly, buttons triggered the correct functions, and all layouts adjusted responsively to various screen sizes. Together, these testing strategies ensured that the final product was both functional and user-focused.

Demo Video
GitHub Repository Link